

Desenvolvimento de Software com IA

Atividade Prática Assíncrona. Relatório final (Etapa 6)

Aluno: Geomerez Raduan de Oliveira Bandeira Professor: Prof. Dr. Jean Mário Moreira de Lima Natal/RN,
agosto de 2026

Repositório: https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona

O texto completo das Etapas 1 a 5, que não cabe nesta página, está em `ia-dev-lab/docs/relatorio-e-tapas.md`. Abaixo estão os quatro pontos pedidos no enunciado e o mapa de onde o professor encontra cada detalhe.

1. Ferramentas e por quê. Eu configurei o Cursor (versão 3.17.8), porque já o uso no dia a dia e tenho licença paga. Ele cobre a IDE com assistente e o agente no próprio editor. Também liguei dois servidores MCP: filesystem, com npx, e GitHub, com a imagem oficial no Docker. Detalhe da escolha: Etapa 1 em `ia-dev-lab/docs/relatorio-etapas.md` e ADR `ia-dev-lab/docs/adr/0001-escolha-do-cursor.md`.

2. Trecho útil do AGENTS.md. O trecho que mais me ajudou foi este, nas convenções: *Regra de negócio na camada de aplicação (Service), não no controller e não na tela*. Ele é útil porque a IA tende a colocar a regra no controller quando o pedido é curto. Esse trecho, junto com a seção Não fazer, foi o que eu cobreí de novo no prompt eficaz da Etapa 4. Arquivo completo: `ia-dev-lab/AGENTS.md`. Regras por pasta: `ia-dev-lab/.cursor/rules/`. Prompts que conferiram o arquivo (subir, testar e parar o Docker): Etapa 2 em `ia-dev-lab/docs/relatorio-etapas.md`.

3. Prompt fraco versus prompt eficaz. O prompt fraco foi só “Adicione arquivar tarefa.” No Grok 4.6 a regra caiu no controller, sem Service e sem teste, e a tarefa arquivada continuou na lista ativa. O prompt eficaz disse o contexto, o padrão (teste primeiro, um Service por caso de uso, controller só HTTP), as restrições e como validar. Os dois modelos então criaram um Service novo e escreveram testes. A diferença maior foi essa: sem dizer onde a regra vive e como conferir, a IA entrega o caminho mais curto. Prompts completos, os quatro casos (Grok e Composer, fraco e eficaz) e os números de teste: `ia-dev-lab/docs/prompts-comparacao.md` e Etapa 4 em `ia-dev-lab/docs/relatorio-etapas.md`.

4. Um obstáculo e como resolvi. O servidor MCP filesystem não subiu, porque esta máquina não tinha npx. Eu instalei o npx no Windows, autorizei os servidores no Cursor e fiz o login do GitHub no navegador. Depois pedi para listar os arquivos do último commit. O MCP do GitHub respondeu: `relatorio.tex` e `relatorio.pdf`. Configuração: `.mcp.json` na raiz. Relato completo: Etapa 5 em `ia-dev-lab/docs/relatorio-etapas.md`.

Onde encontrar o restante.

Etapa 1 (`hello.py`, prompt e execução): `ia-dev-lab/hello.py` e `ia-dev-lab/docs/relatorio-etapas.md`.

Etapa 2 (ToDo Laravel e React, `AGENTS.md`, regras): `ia-dev-lab/AGENTS.md`, `ia-dev-lab/.cursor/rules/` e `ia-dev-lab/docs/relatorio-etapas.md`.

Etapa 3 (pastas, README, ADRs): `ia-dev-lab/README.md` e `ia-dev-lab/docs/adr/` (0001 Cursor, 0002 monorepo, 0003 Docker, 0004 Service).

Etapa 4 (arquivar tarefa): `ia-dev-lab/docs/prompts-comparacao.md`.

Etapa 5 (branch, commit, PR e MCP): branch `feature/setup-inicial`; PR que eu aceitei no GitHub https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona/pull/1; PR aberto https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona/pull/2; arquivo `.mcp.json`.

Laboratório: `ia-dev-lab/backend/` e `ia-dev-lab/frontend/`.